Flask Backend Architecture & Codebase Update Prompt

You are updating an existing **Python Flask backend** for a mobile application.

The developer is coming from a **TypeScript/Node.js/Express.js** background, so the architecture should be clean, modular, predictable, and easy to understand for an Express developer while still following modern Flask/Python conventions.

The application is a **BookMyEvent mobile application backend**.

---

1. Core Requirements

Update the existing backend architecture and codebase according to the following rules.

Technology Stack

Use:

- Python 3.x
- Flask
- SQLAlchemy
- Flask-Migrate / Alembic
- PostgreSQL
- Pydantic for request validation
- Flask-JWT-Extended for JWT authentication
- Flask-CORS
- Redis where required
- AWS Lambda as the eventual deployment target
- AWS RDS PostgreSQL as the production database

The backend must be designed for a **mobile application API**, not a server-rendered web application.

Use RESTful JSON APIs throughout the backend.

---

2. Architecture Philosophy

Use a **feature/domain-based modular architecture**.

Each business domain should be isolated inside:

```
app/modules/
```

Example:

```
app/modules/
├── auth/
├── users/
├── events/
├── bookings/
├── exhibitors/
├── stalls/
├── payments/
├── checkins/
└── admin/
```

Each module should contain its own:

```
routes/
services/
repository/
schemas/
```

Do not put business logic inside routes.

Do not put database queries directly inside routes.

Do not put request validation directly inside routes.

Keep routes thin.

---

3. Final Folder Structure

Use this structure:

```
Backend/
├── run.py
├── main.py
├── requirements.txt
│
├── .env
```

```
├── .env.development
├── .env.production
│
├── migrations/
│   └── versions/
│
├── uploads/
│
├── scripts/
│   └── init_db.py
│
└── app/
    ├── __init__.py
    ├── config.py
    ├── database.py
    ├── extensions.py
    │
    ├── middleware/
    │   ├── auth_middleware.py
    │   └── role_required.py
    │
    ├── exceptions/
    │   ├── api_error.py
    │   └── handlers.py
    │
    ├── utils/
    │   ├── jwt_utils.py
    │   ├── response_utils.py
    │   └── validators.py
    │
    ├── models/
    │   ├── user.py
    │   ├── event.py
    │   ├── category.py
    │   ├── booking.py
    │   ├── stall.py
    │   └── venue.py
    │
    └── modules/
        ├── auth/
        │   ├── __init__.py
        │   ├── routes/
        │   │   └── auth_routes.py
        │   ├── services/
        │   │   └── auth_service.py
        │   ├── repository/
```

```
│   │   └── auth_repository.py
│   └── schemas/
│       └── auth_schema.py
│
├── users/
│   ├── __init__.py
│   ├── routes/
│   │   └── user_routes.py
│   ├── services/
│   │   └── user_service.py
│   ├── repository/
│   │   └── user_repository.py
│   └── schemas/
│       └── user_schema.py
│
├── events/
│   ├── __init__.py
│   ├── routes/
│   │   └── event_routes.py
│   ├── services/
│   │   └── event_service.py
│   ├── repository/
│   │   └── event_repository.py
│   └── schemas/
│       └── event_schema.py
│
├── bookings/
│   ├── __init__.py
│   ├── routes/
│   │   ├── booking_routes.py
│   │   └── seat_routes.py
│   ├── services/
│   │   ├── booking_service.py
│   │   └── seat_service.py
│   ├── repository/
│   │   └── booking_repository.py
│   └── schemas/
│       └── booking_schema.py
│
├── exhibitors/
│   ├── __init__.py
│   ├── routes/
│   │   └── exhibitor_routes.py
│   ├── services/
│   │   └── exhibitor_service.py
│   ├── repository/
```

```
│   │   └── exhibitor_repository.py
│   └── schemas/
│       └── exhibitor_schema.py
│
├── stalls/
│   ├── __init__.py
│   ├── routes/
│   │   └── stall_routes.py
│   ├── services/
│   │   └── stall_service.py
│   ├── repository/
│   │   └── stall_repository.py
│   └── schemas/
│       └── stall_schema.py
│
├── payments/
│   ├── __init__.py
│   ├── routes/
│   │   └── payment_routes.py
│   ├── services/
│   │   └── payment_service.py
│   ├── repository/
│   │   └── payment_repository.py
│   └── schemas/
│       └── payment_schema.py
│
├── checkins/
│   ├── __init__.py
│   ├── routes/
│   │   └── checkin_routes.py
│   ├── services/
│   │   └── checkin_service.py
│   ├── repository/
│   │   └── checkin_repository.py
│   └── schemas/
│       └── checkin_schema.py
│
└── admin/
    ├── __init__.py
    ├── routes/
    │   └── admin_routes.py
    ├── services/
    │   └── admin_service.py
    ├── repository/
    │   └── admin_repository.py
```

```
        └── schemas/
            └── admin_schema.py
```

---

4. IMPORTANT: Route Style

Use the following Flask route style throughout the project.

Do NOT try to imitate Express syntax.

Do NOT use `app.route()` unnecessarily.

Use Flask's modern HTTP-method decorators:

```python
@product_bp.get("/")
def get_all_products():
    return ProductController.get_all_products()


@product_bp.post("/")
@authenticate_token
def create_product():
    return ProductController.create_product()
```

This is the required route style.

For example:

```python
@event_bp.get("/")
def get_all_events():
    return EventController.get_all_events()


@event_bp.get("/<int:event_id>")
def get_event(event_id):
    return EventController.get_event(event_id)


@event_bp.post("/")
@authenticate_token
def create_event():
    return EventController.create_event()
```

```python
@event_bp.put("/<int:event_id>")
@authenticate_token
def update_event(event_id):
    return EventController.update_event(event_id)


@event_bp.delete("/<int:event_id>")
@authenticate_token
def delete_event(event_id):
    return EventController.delete_event(event_id)
```

Keep each route definition readable and separated.

Do not combine multiple routes into one line.

---

5. Controller Layer

Although the previous architecture described routes as the controller equivalent, use an explicit controller layer if the existing codebase already follows a Controller → Service architecture.

Therefore:

```
Route
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
SQLAlchemy
  ↓
PostgreSQL
```

The responsibilities must be strictly separated.

Route

Responsible only for:

- HTTP method
- URL

- authentication decorators
- authorization decorators
- calling controller

Example:

```
@product_bp.get("/")
def get_all_products():
    return ProductController.get_all_products()
```

Controller

Responsible for:

- reading request data
- reading path/query parameters
- calling service
- formatting HTTP response

Example:

```
class ProductController:

    @staticmethod
    def get_all_products():
        products = ProductService.get_all_products()

        return jsonify({
            "success": True,
            "products": products
        }), 200
```

Service

Responsible for:

- business logic
- business validation
- workflows
- calling repositories
- raising domain/API errors

Example:

```
class ProductService:

    @staticmethod
    def get_all_products():
        return ProductRepository.get_all()
```

Repository

Responsible only for:

- database queries
- creating records
- updating records
- deleting records
- fetching records

Example:

```
class ProductRepository:

    @staticmethod
    def get_all():
        return Product.query.all()
```

---

6. Authentication

Use:

```
@authenticate_token
```

as the authentication decorator.

Example:

```
@product_bp.post("/")
@authenticate_token
def create_product():
    return ProductController.create_product()
```

Do not put authentication logic inside controllers.

Do not repeat JWT validation manually inside every controller.

The authentication decorator should:

1. Extract JWT
2. Validate JWT
3. Load/identify the current user
4. Attach the authenticated user/context
5. Reject unauthorized requests

---

7. Role-Based Access Control

Create:

```
app/middleware/role_required.py
```

Use a decorator such as:

```
@role_required(["admin"])
```

Example:

```
@admin_bp.get("/dashboard")
@authenticate_token
@role_required(["admin"])
def get_dashboard():
    return AdminController.get_dashboard()
```

Role authorization must remain separate from business logic.

---

8. Request Validation

Use Pydantic schemas.

Example:

```
from pydantic import BaseModel, Field
```

```python
class CreateEventSchema(BaseModel):
    name: str
    venue_id: int
    category_id: int
```

Controllers/services should validate incoming request data using these schemas.

Do not create large manual validation blocks such as:

```python
if not data.get("name"):
    ...
if not data.get("venue_id"):
    ...
```

Use Pydantic instead.

---

9. Error Handling

Use a custom:

```python
class ApiError(Exception):
    ...
```

Business/application errors should be raised:

```python
raise ApiError("Event not found", 404)
```

Do not write repetitive:

```python
try:
    ...
except Exception:
    return jsonify(...)
```

inside every controller/service.

Use global Flask error handlers.

Example:

```
@app.errorhandler(ApiError)
def handle_api_error(error):
    return jsonify({
        "success": False,
        "message": error.message
    }), error.status_code
```

Also have a generic unexpected-error handler that logs the actual exception but does not expose internal details to the mobile client.

---

10. Response Format

The API is consumed by a mobile application.

Keep responses consistent.

Success:

```
{
    "success": true,
    "data": {}
}
```

List response:

```
{
    "success": true,
    "data": [],
    "pagination": {
        "page": 1,
        "limit": 20,
        "total": 100
    }
}
```

Error:

```
{
    "success": false,
```

```
    "message": "Event not found"
}
```

Do not expose:

- stack traces
- SQL errors
- internal Python exceptions
- database credentials
- internal implementation details

---

11. Database

Use:

```
SQLAlchemy
+
PostgreSQL
+
Flask-Migrate/Alembic
```

Models belong in:

```
app/models/
```

Models are the source of truth for database schema.

Example:

```python
class Event(db.Model):
    __tablename__ = "events"

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(255), nullable=False)
    venue_id = db.Column(
        db.Integer,
        db.ForeignKey("venues.id"),
        nullable=False
    )
```

Never create tables automatically during application startup.

Use migrations:

```
flask db migrate -m "create events table"
flask db upgrade
```

---

12. Repository Rules

Repositories must not contain business rules.

Bad:

```
if event.status != "published":
    ...
```

inside repository code.

Good:

```
Service:
    check event status
    apply business rules
    call repository

Repository:
    execute database query
```

Repositories should be simple and predictable.

---

13. Service Rules

Services contain business logic.

Example:

```
class BookingService:

    @staticmethod
    def create_booking(user_id: int, raw_data: dict):
```

```
        data = CreateBookingSchema(**raw_data)

        if not SeatService.are_seats_available(
            data.event_id,
            data.seat_ids
        ):
            raise ApiError(
                "One or more seats already booked",
                409
            )

        SeatService.lock_seats(
            data.event_id,
            data.seat_ids
        )

        booking = BookingRepository.create(
            user_id=user_id,
            event_id=data.event_id,
            seat_ids=data.seat_ids,
            amount=data.amount
        )

        return booking
```

Keep complex workflows in services, not routes.

---

14. Blueprint Architecture

Each module should expose one module-level blueprint.

Example:

```
# app/modules/events/__init__.py

from flask import Blueprint

from app.modules.events.routes.event_routes import event_bp

events_module_bp = Blueprint(
    "events_module",
    __name__
)
```

```
events_module_bp.register_blueprint(event_bp)
```

Then register it inside the application factory:

```
app.register_blueprint(
    events_module_bp,
    url_prefix="/api/events"
)
```

---

15. App Factory

`app/__init__.py` must use an application factory.

Example:

```
from flask import Flask
from flask_cors import CORS

from app.extensions import db, jwt, migrate
from app.exceptions.handlers import register_error_handlers

from app.modules.auth import auth_module_bp
from app.modules.events import events_module_bp
from app.modules.bookings import bookings_module_bp


def create_app(config_name: str = "development") -> Flask:

    app = Flask(__name__)

    CORS(app)

    db.init_app(app)
    jwt.init_app(app)
    migrate.init_app(app, db)

    register_error_handlers(app)

    app.register_blueprint(
        auth_module_bp,
        url_prefix="/api/auth"
```

```
    )

    app.register_blueprint(
        events_module_bp,
        url_prefix="/api/events"
    )

    app.register_blueprint(
        bookings_module_bp,
        url_prefix="/api/bookings"
    )

    return app
```

Do not create a globally configured Flask application with all logic in one file.

---

16. Main Entry Point

Use:

```
# main.py

import os

from app import create_app


app = create_app(
    os.environ.get(
        "FLASK_ENV",
        "development"
    )
)


if __name__ == "__main__":
    app.run(
        host="0.0.0.0",
        port=5001,
        debug=True
    )
```

Keep application initialization separate from the entry point.

---

17. Python Coding Style

The developer is coming from TypeScript.

Use modern, readable Python.

Prefer:

```
def get_user(user_id: int) -> User:
```

instead of:

```
def get_user(user_id):
```

Use type hints consistently.

Use:

```
list[int]
dict[str, Any]
str
int
bool
```

where appropriate.

Use classes for:

- controllers
- services
- repositories

Use static methods when no instance state is required.

Example:

```
class EventService:

    @staticmethod
    def get_event(event_id: int):
        ...
```

18. Naming Conventions

Use:

```
snake_case
```

for:

- Python files
- functions
- variables
- methods

Examples:

```
event_service.py
booking_repository.py
get_all_events()
create_booking()
```

Use PascalCase for classes:

```
EventService
BookingRepository
CreateBookingSchema
ApiError
```

Blueprint names should remain descriptive:

```
event_bp
booking_bp
auth_bp
```

19. Mobile API Considerations

This backend is specifically for a **React Native mobile application**.

Therefore:

- APIs must return JSON
- No server-side HTML rendering
- Consistent response structures
- Proper HTTP status codes
- JWT authentication
- Pagination for large lists
- Filtering and sorting where required
- Proper validation
- Mobile-friendly error messages
- Secure authentication
- Refresh-token strategy where required
- Rate limiting for sensitive endpoints
- CORS configuration
- File upload support where required
- Presigned S3 uploads for large media/files where appropriate

---

20. API Versioning

Use:

```
/api/v1/
```

as the API prefix.

Example:

```
GET    /api/v1/events
GET    /api/v1/events/123
POST   /api/v1/events
PUT    /api/v1/events/123
DELETE /api/v1/events/123
```

The mobile application should never depend on unversioned API endpoints.

---

21. Example Final Route File

The final route implementation should look like this:

```python
# app/modules/events/routes/event_routes.py

from flask import Blueprint

from app.modules.events.controllers.event_controller import (
    EventController
)

from app.middleware.auth_middleware import authenticate_token


event_bp = Blueprint(
    "events",
    __name__
)


@event_bp.get("/")
def get_all_events():
    return EventController.get_all_events()


@event_bp.get("/<int:event_id>")
def get_event(event_id: int):
    return EventController.get_event(event_id)


@event_bp.post("/")
@authenticate_token
def create_event():
    return EventController.create_event()


@event_bp.put("/<int:event_id>")
@authenticate_token
def update_event(event_id: int):
    return EventController.update_event(event_id)


@event_bp.delete("/<int:event_id>")
@authenticate_token
def delete_event(event_id: int):
    return EventController.delete_event(event_id)
```

This exact style should be followed throughout the entire backend.

---

22. Example Controller

```python
# app/modules/events/controllers/event_controller.py

from flask import jsonify, request

from app.modules.events.services.event_service import EventService


class EventController:

    @staticmethod
    def get_all_events():

        events = EventService.get_all_events()

        return jsonify({
            "success": True,
            "data": events
        }), 200


    @staticmethod
    def get_event(event_id: int):

        event = EventService.get_event(event_id)

        return jsonify({
            "success": True,
            "data": event
        }), 200


    @staticmethod
    def create_event():

        data = request.get_json() or {}

        event = EventService.create_event(data)

        return jsonify({
            "success": True,
            "data": event
        }), 201
```

Do not place database queries inside this controller.

---

23. Example Service

```
# app/modules/events/services/event_service.py

from app.exceptions.api_error import ApiError
from app.modules.events.repository.event_repository import (
    EventRepository
)
from app.modules.events.schemas.event_schema import (
    CreateEventSchema
)


class EventService:

    @staticmethod
    def get_all_events():

        events = EventRepository.get_all()

        return [
            event.to_dict()
            for event in events
        ]


    @staticmethod
    def get_event(event_id: int):

        event = EventRepository.get_by_id(event_id)

        if not event:
            raise ApiError(
                "Event not found",
                404
            )

        return event.to_dict()


    @staticmethod
    def create_event(raw_data: dict):
```

```
        data = CreateEventSchema(
            **raw_data
        )

        event = EventRepository.create(
            data
        )

        return event.to_dict()
```

---

24. Example Repository

```
# app/modules/events/repository/event_repository.py

from app.extensions import db
from app.models.event import Event


class EventRepository:

    @staticmethod
    def get_all() -> list[Event]:

        return Event.query.all()


    @staticmethod
    def get_by_id(event_id: int) -> Event | None:

        return Event.query.get(event_id)


    @staticmethod
    def create(data):

        event = Event(
            name=data.name,
            venue_id=data.venue_id,
            category_id=data.category_id
        )

        db.session.add(event)
        db.session.commit()
```

```
        return event
```

---

25. Do Not Over-Engineer

Do not introduce unnecessary abstractions.

Do not create:

```
factories/
managers/
handlers/
adapters/
use_cases/
providers/
```

unless there is a real requirement.

The core flow should remain:

```
Route
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Model / SQLAlchemy
  ↓
PostgreSQL
```

Authentication/authorization:

```
Route
  ↓
authenticate_token
  ↓
role_required
```

```
  ↓
Controller
```

Errors:

```
Service
  ↓
raise ApiError
  ↓
Global Error Handler
  ↓
JSON Response
```

---

26. AWS Lambda Compatibility

The architecture must remain compatible with AWS Lambda.

Avoid:

- long-running background processes
- filesystem-dependent persistent storage
- in-memory application state
- local database assumptions

Uploaded files should ultimately use S3.

PostgreSQL should use AWS RDS.

Redis should be external, such as ElastiCache or another managed Redis provider.

The application should remain stateless.

---

27. Environment Configuration

Do not hardcode:

- database URLs
- JWT secrets
- Redis URLs
- AWS credentials
- Razorpay keys

- Firebase credentials
- API keys

Use environment variables.

Example:

```
DATABASE_URL=
JWT_SECRET_KEY=
REDIS_URL=
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_REGION=
S3_BUCKET_NAME=
```

Use:

```
app/config.py
```

for configuration management.

---

28. Requirements

Maintain dependencies in:

```
requirements.txt
```

At minimum, evaluate the need for:

```
Flask
Flask-SQLAlchemy
Flask-Migrate
Flask-JWT-Extended
Flask-CORS
Pydantic
psycopg2-binary
redis
python-dotenv
```

Pin versions appropriately for production.

Do not add dependencies without a clear reason.

---

29. Migration Rules

Never modify production database schema manually.

Use:

```
flask db migrate -m "description"
flask db upgrade
```

Migration files belong in:

```
migrations/versions/
```

Do not manually edit generated migration files unless necessary.

---

30. Final Refactoring Task

Now inspect the existing BookMyEvent Flask backend and refactor it to follow this architecture.

Important:

1. Preserve existing functionality.
2. Do not remove existing API behavior unnecessarily.
3. Do not change API contracts unless required.
4. Move business logic out of routes.
5. Move database queries out of controllers.
6. Move business rules into services.
7. Move database access into repositories.
8. Move validation into Pydantic schemas.
9. Use Flask Blueprint modules.
10. Use the exact route decorator style defined above.
11. Use `@authenticate_token` for protected routes.
12. Use `@role_required(...)` for role-specific routes.
13. Use global error handling.
14. Use SQLAlchemy models.
15. Use Flask-Migrate/Alembic.
16. Keep the backend stateless and Lambda-compatible.
17. Keep the API optimized for the React Native mobile application.

18. Do not introduce unnecessary architectural layers.
19. Use modern Python type hints.
20. Keep route files extremely thin and readable.

---

31. Most Important Route Rule

Every route should follow this pattern:

```
@resource_bp.get("/")
def get_resources():
    return ResourceController.get_resources()


@resource_bp.post("/")
@authenticate_token
def create_resource():
    return ResourceController.create_resource()
```

For protected routes:

```
@resource_bp.put("/<int:resource_id>")
@authenticate_token
def update_resource(resource_id: int):
    return ResourceController.update_resource(resource_id)
```

For role-protected routes:

```
@admin_bp.get("/dashboard")
@authenticate_token
@role_required(["admin"])
def get_dashboard():
    return AdminController.get_dashboard()
```

Do not use Express-style route registration.

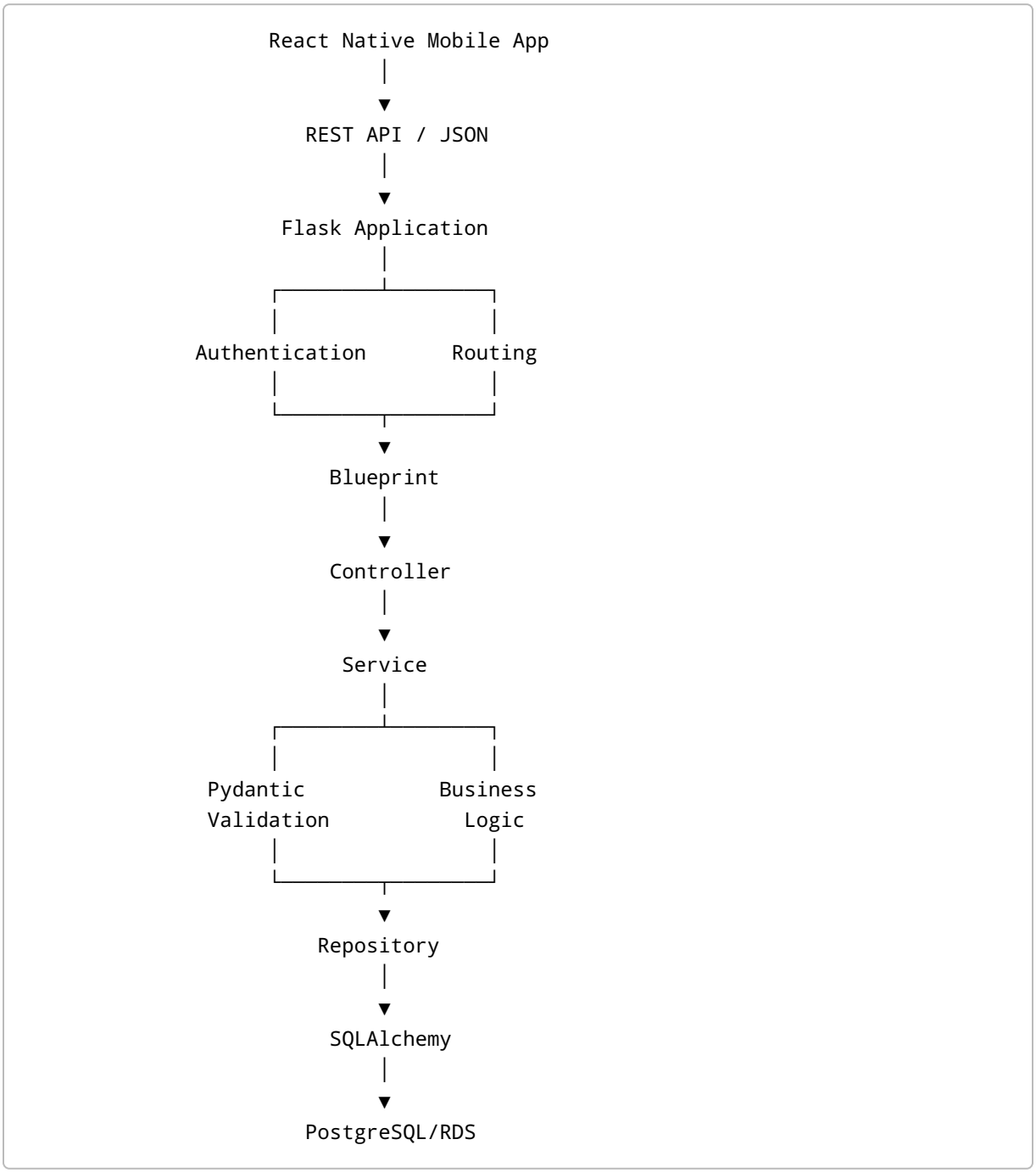
Do not put service calls directly in route functions.

Always call the controller from the route.

---

32. Final Expected Architecture

The final application should clearly follow:



Keep this architecture consistent across every feature.

Do not create a different pattern for individual modules.

The goal is a clean, production-ready Flask backend that feels familiar to a TypeScript/Express developer while following proper Python/Flask architecture.